

Table des matières

Documentation détaillée — Blackjack POO	1
1. Contexte et besoin	1
2. Fonctionnalités	1
3. Choix techniques	2
4. Détails d'implémentation notables	3
5. Tests	3
6. Utilisation de l'IA	4
7. Données nécessaires	5
8. Lancement et utilisation	5
9. Limites connues	5
10. Bibliographie	5

Documentation détaillée — Blackjack POO

1. Contexte et besoin

Projet personnel rendu dans le cadre du module **NFP01 – Programmation Orientée Objet en Python, Java et autres** (CNAM, 2025-2026, M. Adrien Escourrou).

Le sujet est libre, sous trois contraintes : logique significative, projet entièrement fini, et exigences minimales (modules, I/O, tests, documentation). Le sujet retenu — un **jeu de Blackjack** — coche toutes ces cases tout en offrant un terrain idéal pour démontrer l'héritage, le polymorphisme et l'encapsulation.

Au-delà du jeu lui-même, l'objectif pédagogique est de **rendre visibles toutes les méthodes de prise de décision existantes** : stratégie de base et systèmes de comptage de cartes.

2. Fonctionnalités

Centrale

Jeu de Blackjack en CLI, actions standard (Hit, Stand, Double, Split, Surrender) plus l'assurance, règles paramétrables.

Plus-value pédagogique

- **Aide à la décision** activable au lancement (désactivée par défaut) : le joueur sélectionne l'une des **neuf stratégies** disponibles (1 manuelle, 1 de base, 7 comptages) et le conseil s'affiche à chaque tour, sans l'obliger à le suivre.
- **Mode Didacticiel** (commande 2 du menu) : pensé pour les débutants, tout est commenté pas à pas par le croupier (narration), le conseil de la stratégie de base est affiché et chaque action possible est expliquée, avec animations.

Bonus

- Mode **simulation comparative** (commande 4 du menu) : un robot joue N manches par stratégie et l'EV empirique est comparée ; les comptages misent davantage quand le sabot est favorable (mise variable selon le *true count*).
- **Sauvegarde & reprise** : solde, statistiques cumulées et records persistés dans `~/black-jack_profile.json` ; **re-cave** en cas de faillite ; **séries de victoires** et **records** affichés.
- **Animations** (distribution carte par carte, suspense) et **narration** activables.

- **Musique d'ambiance** (commande 6) : morceau de lounge généré hors-ligne ou webradio (flux So-maFM), via un lecteur audio système.
 - **Journalisation** dans `logs/blackjack.log`.
 - **Configuration** des règles dans `config/default.json`.
-

3. Choix techniques

Langage et environnement

Python 3.10+ (annotations de type, `from __future__ import annotations, dataclasses, énumérations`).

Dépendances minimales : seules `rich` (panneaux, tables, couleurs RGB) et `pyfiglet` (gros titres ASCII) sont requises, toutes deux pour l'interface CLI ; le moteur de jeu reste 100 % bibliothèque standard. La musique d'ambiance s'appuie sur un lecteur audio déjà présent sur le système (`afplay`, `ffplay`, `aplay...`), sans paquet supplémentaire.

Architecture en couches

ui/ (CLI, prompts, affichage)
game/ (Rules, Round, Game, Statistics, Profile)
players/ (BasePlayer → Dealer / Human) strategies/ (Strategy → 9 stratégies)
core/ (Card, Hand, Shoe, énumérations)

Chaque couche ne dépend que des couches inférieures. La couche `ui` est injectée dans `HumanPlayer` au moment de la création de `Game`, ce qui permettrait par la suite de remplacer la CLI par une GUI sans toucher au moteur.

Patrons de conception utilisés

- **Strategy pattern** : classe `Strategy` abstraite et 9 implémentations interchangeable. C'est le polymorphisme de l'enseignement traduit en code.
- **Template method** : `BasePlayer.decide()` reste abstrait, chaque sous-classe l'implémente différemment.
- **Registry** : `strategies.STRATEGIES` (dict) sert de registre central des stratégies disponibles — ajouter une stratégie revient à ajouter une ligne.
- **Dataclass** : `Rules` et `Statistics` (réduction du boilerplate).

Encapsulation

Toutes les classes utilisent des attributs privés (`__name`, `__rank`, `__bankroll`, etc.) exposés en lecture via `@property`, et en écriture uniquement par des méthodes publiques validantes (`credit`, `debit`, `add_card`, etc.). Cf. cours « 3 - Héritage Polymorphisme & Encapsulation en Python ».

Sources des valeurs de stratégie

Les valeurs de chaque comptage et le contenu des tables de stratégie de base proviennent d'ouvrages de référence :

- E. O. Thorp, *Beat the Dealer*, 1962 (édition révisée).
- S. Wong, *Professional Blackjack*.
- D. Schlesinger, *Blackjack Attack*.
- A. Snyder, *Blackbelt in Blackjack* (Zen Count, Red 7).
- B. Carlson, *Blackjack for Blood* (Omega II).
- O. Vancura & K. Fuchs, *Knock-Out Blackjack* (KO).

Ces valeurs ont été croisées avec plusieurs sources web spécialisées (blackjackhero.com, casinoguardian.co.uk, lolblackjack.com, blackjackstrategist.com, etc.).

4. Détails d'implémentation notables

Calcul du total d'une main (`Hand.total`)

On somme toutes les cartes avec l'As compté 11, puis on ramène chaque As à 1 (`total -= 10`) tant que le total dépasse 21. Cette boucle gère proprement le cas des **deux As** dans la même main.

Distinction soft / hard

Une main est **soft** quand elle contient un As compté 11 dans le total courant. C'est crucial pour la stratégie de base (tableau dédié aux mains soft).

Sabot avec carte de coupe

Le Shoe n'est jamais remélangé en plein milieu d'une main. Quand la pénétration est atteinte, un drapeau `needs_shuffle` est levé ; le moteur (`Game.play_round`) le consulte **avant** chaque manche et remélange seulement à ce moment-là.

Comptage et carte cachée du croupier

Subtilité importante : la carte cachée (hole card) du croupier n'a pas été *visible* du joueur, donc le compteur ne doit pas la comptabiliser tant qu'elle reste cachée. Le code annule explicitement l'observation faite au moment de la distribution, puis l'observe à nouveau au moment du retournement. Sans cette précaution, les comptages seraient systématiquement biaisés.

Split et règles spécifiques

- Une seule carte est tirée après split d'As (règle classique).
 - Le ré-split d'As est interdit par défaut (paramétrable).
 - Le nombre maximum de mains après splits est paramétrable (`max_splits`).
 - Une main issue d'un split ne peut **pas** être un blackjack naturel (drapeau `from_split` posé).
-

5. Tests

9 fichiers, 87 cas de test au total. Couverture : - Cartes : création, immutabilité, égalité, hash. - Sabot : reproductibilité, taille, signal de remélange, brûlage de carte. - Mains : soft/hard, As multiples, paires, bust, double. - Stratégies : décisions canoniques, valeurs de tous les comptages, équilibre (somme = 0) des comptages balanced. - Manche : règlement des gains (blackjack 3:2, push, abandon, bust, égalité), restriction de double aux durs 9-11, flux carte cachée + peek, assurance, garde anti-boucle sur le split. - Profil : sauvegarde/reprise JSON, cumul et records, séries de victoires. - Simulation : mise variable selon le true count. - Audio : génération du morceau, playlist/webradio, mode dégradé. - End-to-end : 200 manches simulées sans exception.

```
python -m unittest discover -s tests -v
```

6. Utilisation de l'IA

Déclaration conforme à la section 3 du sujet. Cette section est volontairement détaillée et honnête : elle décrit franchement l'ampleur de l'usage de l'IA, ce que l'étudiant a dirigé, compris et validé, ainsi que le taux d'utilisation.

Outil utilisé

- **Claude (Anthropic)**, via l'assistant **Claude Code** (pair-programming en ligne de commande, modèles Claude Opus / Sonnet selon les étapes).
- Aucun autre outil d'IA générative n'a été utilisé.

Ampleur réelle / taux d'utilisation

Le projet a été développé en **binôme avec l'IA, de bout en bout** : l'étudiant définit chaque besoin et pilote l'itération ; l'IA propose le code, explique et exécute les tests. Pour être totalement transparent : - **Taux d'utilisation** : ~100 %. L'**intégralité du code source** (moteur de jeu, interface, stratégies de comptage, persistance, audio, tests) ainsi que la **documentation et le README** ont été générés ou réécrits avec l'IA. - **Tout** a néanmoins été réalisé **sous la direction explicite de l'étudiant** (voir « Pilotage » ci-dessous) : aucune fonctionnalité n'a été ajoutée sans demande, et chaque proposition a été relue, testée et validée — ou refusée.

Pourquoi l'IA a été utilisée

- **Génération** de modules complets (cartes, mains, sabot, stratégies, UI).
- **Refactorisation** (ex. factoriser le rendu des cartes, source unique du palier de mise BET_RAMP).
- **Débogage** (ex. correction d'une **boucle infinie** sur le split au maximum de mains, d'un blocage de la simulation, du double affichage des cartes).
- **Génération de tests** unitaires et de scénarios reproductibles.
- **Revue de code** (passes de relecture ciblées « correctness ») et **vérification croisée** des tables de stratégie et des valeurs de comptage.
- **Rédaction** des docstrings, du README et de cette documentation.

Pilotage par l'étudiant (ce que j'ai dirigé)

Chaque évolution provient d'une demande précise de ma part, par exemple : amélioration de l'interface pour débutants, mode **Didacticiel**, **musique d'ambiance** (puis webradio), **touches d'action en français**, **mise en jetons** sans centime, explication du **palier de mise** selon le true count, **sauvegarde/reprise** et **records**. J'ai aussi tranché des choix de règles (ex. conserver le paiement **3:2**, garder la mise à plat pour la stratégie de base car varier la mise sans comptage est sous-optimal).

Ce que j'ai compris, modifié et validé

- **Compris** : règles du Blackjack (peek, ENHC, S17, double restreint, assurance), principe des comptages (running count -> true count -> palier de mise), et les 3 piliers POO mis en œuvre.
- **Validé** : par le jeu (tests manuels à chaque étape) et par la suite automatisée (python -m unittest, 94 tests). J'ai accepté ou **rejeté** des propositions de l'IA (ex. refus d'arrondir les gains, choix de l'ordre du menu, disposition des panneaux à l'écran).
- **Modifié** : ajustements d'ergonomie et de formulation, choix des valeurs de jetons, position des messages d'aide, etc.

Exemples de prompts réellement utilisés

- « Améliore l'interface du jeu pour qu'elle soit compréhensible par des joueurs débutants et facile à jouer. »
- « J'aimerais que la musique d'ambiance soit à l'ambiance d'un casino, calme et agréable... il n'existerait pas des webradios lounge ? »
- « Les touches des actions devraient être en français (t pour tirer, etc.). »
- « Rajoute dans le tableau des stratégies à partir de quand augmenter les mises, de combien, et pourquoi. »
- « J'ai eu un blackjack et le croupier a quand même tiré une carte. » (rapport de bug -> correction).
- « Regarde tout le code et, quand l'utilisateur doit décider, rends l'info claire pour un débutant. »
- (commande de revue) « /code-review » pour une relecture orientée bugs.

Marquage du code généré par IA

Le **code étant généré à ~100 % par l'IA**, encadrer chaque ligne par des marqueurs # CODE IA reviendrait à marquer tout le projet et nuirait à la lisibilité. Le choix retenu — qui couvre donc l'**ensemble des fichiers** — est une **déclaration globale** : tout le code source du projet est généré avec Claude (Claude Code), l'étudiant en assurant la conception, la direction, la relecture et la validation, et restant **capable d'expliquer chaque partie devant le jury**.

7. Données nécessaires

Aucune donnée externe n'est nécessaire pour faire fonctionner le projet. Le fichier `config/default.json` est fourni avec des valeurs raisonnables et peut être édité.

8. Lancement et utilisation

Voir le `README.md` à la racine.

9. Limites connues

- Pas d'**index plays** (déviations de décision selon le true count) : les comptages décident comme la stratégie de base et ne se distinguent que par la mise variable.
 - Pas d'**even money** sur blackjack joueur face à un As (l'assurance, elle, est gérée).
 - Pas de mode **multi-joueurs**.
 - Pas d'interface graphique (CLI uniquement).
-

10. Bibliographie

- E. O. Thorp, *Beat the Dealer*, 1962 (révisions ultérieures).
- S. Wong, *Professional Blackjack*.
- D. Schlesinger, *Blackjack Attack*.
- A. Snyder, *Blackbelt in Blackjack*.
- B. Carlson, *Blackjack for Blood*.
- O. Vancura, K. Fuchs, *Knock-Out Blackjack*.
- Documentation Python officielle (<https://docs.python.org/3/>).
- Cours NFP01 (A. Escourrou), supports 1 à 3.